

Chapter 3

Operators and Expressions

Introduction

- An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations.
- Operators are used in programs to manipulate data and variables.
- C operators can be classified into a number of categories.
 - Arithmetic operators
 - Relational operators
 - Logical operators
 - Assignment operators
 - Increment and decrement operators
 - Conditional operators
 - Bitwise operators
 - Special operators
- An expression is a sequence of operands and operators that reduces to a single value.

Example : 10+15

Arithmetic Operators

Operator	meaning
+	Addition or unary plus
-	Subtraction or unary minus
*	Multiplication
/	Division
%	modulo division

Arithmetic operators

- Examples of arithmetic operators are:

$a + b$

$a - b$

$a * b$

a / b

$a \% b$

Where a and b are knowns as operands.

Integer Arithmetic

- When both operands of expression are of integer types, then the expression known as integer arithmetic.
- Suppose $a=14$ and $b=4$
 - $a + b = 18$
 - $a - b = 10$
 - $a * b = 56$
 - $a / b = 3$ (Quotient)
 - $a \% b = 2$ (Reminder)

Integer Arithmetic

- Examples

$$6/7 = 0 \text{ and } -6/-7 = 0$$

but $-6/7$ may be 0 or -1 (machine dependent)

- Examples

$$-14 \% 3 = -2$$

$$-14 \% -3 = -2$$

$$14 \% -3 = 2$$

Note: In modulo division, the sign of result is always the sign of the first operand.

Real Arithmetic

- An arithmetic operations involving only real operands is called as real arithmetic.
- Examples

$$x = 6.0 / 7.0 = 0.857143$$

$$y = -2.0 / 3.0 = -0.666667$$

Note: The modulo operator (%) cannot be used with real operands.

Mixed-mod Arithmetic

- When one operand is real and the other is integer, the expression is called a mixed-mode arithmetic.
- Examples

$$15/10.0 = 1.5$$

$$15/10 = 1$$

Relational Operators

Operator	Meaning
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	equal to
!=	not equal to

Relational Operators

- A relational expression yields a value of 1 or 0.

5 < 6

TRUE

logic 1

6 < 5

FALSE

logic 0

Relational Operators Complements

$>$ is complement of $\leq$
 $<$ is complement of $\geq$
 $==$ is complement of $\neq$

- $!(x < y) \rightarrow x \geq y$
- $!(x > y) \rightarrow x \leq y$
- $!(x \neq y) \rightarrow x == y$
- $!(x \leq y) \rightarrow x > y$
- $!(x \geq y) \rightarrow x < y$
- $!(x == y) \rightarrow x \neq y$

Logical Operators

- C has the following three logical operators

&& meaning logical and

|| meaning logical or

! meaning logical not (unary operator)

- Examples

a > b && a == 2 True When a=2 and b=1

a > b || a == 1 True When a=2 and b=1

Logical Operators

Table 3.3 Truth Table

op-1	op-2	Value of the expression	
		op-1 && op-2	op-1 op-2
Non-zero	Non-zero	1	1
Non-zero	0	0	1
0	Non-zero	0	1
0	0	0	0

Assignment Operators

- Assignment operators are used to assign the result of an expression to a variable.

`c = 2 + 3 ;`

`c = a + b;`

`c = 5;`

- “Shorthand” assignment operators:

`a = a + b ;    a += b;`

`a = a * 2 ;    a *= 2;`

Shorthand Operators

Table 3.4 *Shorthand Assignment Operators*

<i>Statement with simple assignment operator</i>	<i>Statement with shorthand operator</i>
$a = a + 1$	$a += 1$
$a = a - 1$	$a -= 1$
$a = a * (n+1)$	$a *= n+1$
$a = a / (n+1)$	$a /= n+1$
$a = a \% b$	$a \% = b$

Increment and decrement Operators

- C provides two unusual operators for incrementing and decrementing variables.
- The increment operator ++ adds 1 to its operand, while the decrement operator -- subtracts 1.
- The unusual aspect is that ++ and -- may be used either as prefix operators (before the variable, as in ++n), or postfix operators (after the variable: n++).
- In both cases, the effect is to increment n. But the expression ++n increments n *before* its value is used, while n++ increments n *after* its value has been used.

++ m or m ++;

-- m or m --;

++m is equivalent to m = m +1 (or m +=1)

--m is equivalent to m = m -1 (or m -=1)

Increment and decrement Operators

- Examples :
- Suppose $a = 5$ and $b = 4$
- Evaluate the following expressions:

1) $++a$

2) $b++$

3) $c = --a + ++b$

4) $d = a++ - b--$

Conditional Operator

- A ternary operator pair “? : ” is available in C to construct conditional expressions of the form

expr1 ? expr2 : expr3

- The expression *expr1* is evaluated first. If it is non-zero (true), then the expression *expr2* is evaluated, and that is the value of the conditional expression. Otherwise *expr3* is evaluated, and that is the value. Only one of *expr2* and *expr3* is evaluated.
- Example:

a=10

b=15

x=(a>b) ? a: b;

Bitwise Operators

Table 3.5 *Bitwise Operators*

Operator	Meaning
&	bitwise AND
	bitwise OR
^	bitwise exclusive OR
<<	shift left
>>	shift right

Bitwise Operators

- Example:

```
int main()
{
    int a =8;           // binary of 8 =01000
    printf("%d\n",a<<1); //sifth left :10000
    printf("%d",a>>1);  //shift right : 00100
    return 0;
}
```

Output:

16

4

Special Operators

- **The Comma Operator:** The comma operator can be used to link the related expressions together. A comma-linked list of expressions is evaluated left to right and the value of right-most expression is the value of the combined expression.

- For example,

`value = (x=10, y=5, x + y);`

first assigns the value 10 to x, then assigns 5 to y, and finally assigns 15 to value. Since comma operator has the **lowest precedence** of all operators, the parentheses are necessary.

Special Operators

- **The sizeof Operator:** The sizeof operator gives the number of bytes the operand occupies.
- The operand can be a variable, a constant and a data type qualifier.
- For example,

```
m = sizeof(x);  
n = sizeof(int);  
p = sizeof(12);  
q = sizeof(12L);
```

Arithmetic Expressions

- An arithmetic expression is a combination of variables, constants, and operators.
- For example,

$$a*b-c \rightarrow a*b-c$$

$$(m+n)(x+y) \rightarrow (m+n)*(x+y)$$

$$ax^2+bx+c \rightarrow a*x*x+b*x+c$$

Evaluate the Expressions

- Suppose $a = 9$, $b = 12$ and $c = 3$

$$X = a - b / 3 + c * 2 - 1 ;$$

$$Y = a - b / (3 + c) * (2 - 1);$$

$$Z = a - (b / (3 + c) * 2) - 1;$$

Type conversions in Expressions

1. Implicit Type Conversion

- C permits mixing of constants and variables of different types in an expression. C automatically converts any intermediate values to the proper type so that the expression can be evaluated without losing any significance. This automatic conversion is known as **implicit type conversion**.
- The rule of type conversion: the **lower** type is automatically converted to the **higher** type.
- Examples:
 - 1) `int x=25; float y=x;`
 - 2) `int x = 'A' + 32;`
 - 3) `float x=2.5; int x = y;`

Type Conversions in Expressions

2. Explicit Type Conversion

- There are some instances when we want to force a type conversion in a way that is different from the automatic conversion.
- The general form is :

(type-name)expression;

- Examples:

int x = 5 / 2 ; → x=2

float x = 5 / 2 ; → x=2.000000

x = (float) 5 / 2;

Explicit type Conversions

✓ **Table 3.7** Use of Casts

Example	Action
<code>x = (int) 7.5</code>	7.5 is converted to integer by truncation.
<code>a = (int) 21.3/(int)4.5</code>	Evaluated as 21/4 and the result would be 5.
<code>b = (double)sum/n</code>	Division is done in floating point mode.
<code>y = (int) (a+b)</code>	The result of a+b is converted to integer.
<code>z = (int)a+b</code>	a is converted to integer and then added to b.
<code>p = cos((double)x)</code>	Converts x to double before using it.

Operator Precedence and Associativity

Table 3.8 Summary of C Operators

Operator	Description	Associativity	Rank
()	Function call	Left to right	1
[]	Array element reference		
+	Unary plus	Right to left	2
-	Unary minus		
++	Increment		
--	Decrement		
!	Logical negation		
~	Ones complement		
*	Pointer reference (indirection)		
&	Address		
sizeof	Size of an object		
(type)	Type cast (conversion)		
*	Multiplication	Left to right	3
/	Division		
%	Modulus		
+	Addition	Left to right	4
-	Subtraction		
<<	Left shift	Left to right	5
>>	Right shift		
<	Less than	Left to right	6
<=	Less than or equal to		
>	Greater than		
>=	Greater than or equal to		
==	Equality	Left to right	7
!=	Inequality		
&	Bitwise AND	Left to right	8
^	Bitwise XOR	Left to right	9
	Bitwise OR	Left to right	10
&&	Logical AND	Left to right	11
	Logical OR	Left to right	12
?:	Conditional expression	Right to left	13
=	Assignment operators	Right to left	14
* = /= % =			
+ = - = & =			
^ = =			
<< = >> =			
,	Comma operator	Left to right	15

Predict the output for following expressions:

Assume a= 5 and b = 3 and c= 4

1. `!(a > b)`
2. `!(a != b)`
3. `a > b && !(b == c)`
4. `a <= c || !(a < b)`
5. `X= (a!=b) ?(a > b) : (b > c)`

Programs

1. Write a C program to convert a given number of days into months and days.
2. Write a C program to swap the value of two integers using following methods:
 - i) With using third variable
 - ii) Without using third variable
3. Write a C program to find maximum value from entered three numbers.
4. Write a C program that reads a floating point number and then displays the right most digit of the integral part of the number.(Hint: if number=15.4 then print 15 and if number = 15.6 then print 16)
5. Write a C program that reads the temperature in Fahrenheit and convert into Celsius.

ANY QUESTIONS?

THANK YOU